

Experiment # 08

Understanding of BFS and DFS algorithms

Objective/s:

- What is BFS and DFS
- Implementation of BFS and DFS
- Lab Tasks

Theory:

BFS stands for Breadth-First Search. It is a graph traversal algorithm used in computer science for traversing or searching tree or graph data structures. The purpose of BFS is to systematically visit and explore all the nodes of a graph or tree by exploring all the neighbor nodes at the present depth level before moving on to the nodes at the next depth level.

BFS starts at a given node (often referred to as the "root" node) and explores all of its neighboring nodes at the same depth level before moving on to the nodes at the next level. This process continues until all nodes have been visited.

The main purpose of BFS is to visit or search for nodes in a graph or tree in a systematic manner, ensuring that all nodes are visited exactly once and in a specific order. BFS is often used to find the shortest path between two nodes in an unweighted graph, to check for connectivity in a graph, or to perform level-order traversal of a tree. It's also used in various applications such as network routing protocols, web crawling, and social network analysis.

BFS Algorithm Steps:

1. Initialize Data Structures:

- Create an empty queue to store vertices to visit.
- Create a set or array to keep track of visited vertices.

2. Enqueue the Starting Vertex:

- Enqueue the starting vertex into the queue.
- Mark the starting vertex as visited.

3. While the Queue is not Empty:

- Dequeue a vertex from the front of the queue.
- Process the dequeued vertex.
- Enqueue all unvisited neighbors of the dequeued vertex.

- Mark each visited neighbor as visited and enqueue it into the queue.

4. Repeat Step 3 Until Queue is Empty:

- Continue the process until the queue becomes empty, indicating that all reachable vertices have been visited.

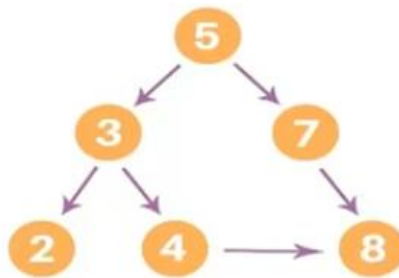
5. Finish:

- Once the queue becomes empty, BFS traversal is complete, and all reachable vertices from the starting vertex have been visited.

6. Optional:

- If there are disconnected components in the graph (i.e., vertices that cannot be reached from the starting vertex), repeat steps 2-5 for each unvisited vertex until all vertices are visited.

Example



Graph representation using adjacency list

```
graph = {  
    '5': ['3', '7'],  
    '3': ['2', '4'],  
    '7': ['8'],  
    '2': [],  
    '4': ['8'],  
    '8': []  
}
```

List to keep track of visited nodes

```
visited = []
```

Queue for BFS

```
queue = []
```

Function for BFS

```
def bfs(visited, graph, node):  
    visited.append(node)  
    queue.append(node)
```

```
while queue:
    m = queue.pop(0)
    print(m, end=" ")

    for neighbour in graph[m]:
        if neighbour not in visited:
            visited.append(neighbour)
            queue.append(neighbour)
```

```
# Driver Code
print("Following is the Breadth-First Search:")
bfs(visited, graph, '5')
```

```
visited = []
queue = []

visited.append(node)
queue.append(node)
visited = [5]
queue = [5]
while queue:
    m = queue.pop(0)
    queue = []
    print (m, end = " ")
    5
    for neighbour in graph[m]:
        if neighbour not in visited:
            visited.append(neighbour)
            visited = [5 3 7]
            queue.append(neighbour)
            queue = [3 7]
```

```
queue = [3 7]
```

```
while queue:
    m = queue.pop(0)
    queue = [7]
    print (m, end = " ")
    5 3
    for neighbour in graph[m]:
        if neighbour not in visited:
            visited.append(neighbour)
            visited = [5 3 7 2 4]
            queue.append(neighbour)
            queue = [7 2 4]
```

queue = [7 2 4]

```
while queue:
m = queue.pop(0)
    queue = [2 4]
print (m, end = " ")
    5 3 7
for neighbour in graph[m]:
    if neighbour not in visited:
visited.append(neighbour)
    visited = [5 3 7 2 4 8]
queue.append(neighbour)
    queue = [2 4 8]
```

queue = [2 4 8]

```
while queue:
m = queue.pop(0)
    # queue = [4 8]
print (m, end = " ")
    # 5 3 7 2
for neighbour in graph[m]:
    if neighbour not in visited:
visited.append(neighbour)
    # visited = [5 3 7 2 4 8]
queue.append(neighbour)
    # queue = [4 8]
```

queue = [4 8]

```
while queue:
m = queue.pop(0)
    queue = [8]
print (m, end = " ")
    5 3 7 2 4
for neighbour in graph[m]:
    if neighbour not in visited:
visited.append(neighbour)
    visited = [5 3 7 2 4 8]
queue.append(neighbour)
    queue = [8]
```

queue = [8]

```
while queue:
m = queue.pop(0)
    queue = []
print (m, end = " ")
    5 3 7 2 4 8
for neighbour in graph[m]:
    if neighbour not in visited:
visited.append(neighbour)
    visited = [5 3 7 2 4 8]
queue.append(neighbour)
    queue = []
```

DFS Algorithm:

DFS stands for Depth-First Search. It is another graph traversal algorithm used in computer science to traverse or search tree or graph data structures. Unlike BFS, which systematically explores all neighbor nodes at the present depth level before moving on to the nodes at the next depth level, DFS explores as far as possible along each branch before backtracking.

The purpose of DFS is to visit and explore deeper into the graph or tree structure before exploring sibling nodes. It goes as deep as possible along each branch before backtracking to explore other

branches. DFS has several applications including:

1. **Traversal:** DFS can be used to traverse all the vertices of a graph or tree. It's commonly used in algorithms like topological sorting and finding strongly connected components in a graph.
2. **Pathfinding:** DFS can be used to find a path between two vertices in a graph, although it doesn't guarantee finding the shortest path (unless the graph is a tree or has uniform edge weights).
3. **Cycle detection:** DFS can be used to detect cycles in a graph. If during the traversal, a back edge (an edge from a descendant to an ancestor in the DFS tree) is found, then the graph contains a cycle.
4. **Maze generation and solving:** DFS can be used to generate mazes and also to solve them.

DFS is often implemented using recursion, but it can also be implemented iteratively using a stack data structure. It's important to note that the choice between DFS and BFS depends on the specific problem requirements and characteristics of the graph or tree being traversed.

DFS algorithm Steps:

1. **Initialize Data Structures:**
 - Create a stack (or use recursion with the call stack) to store vertices to visit.
 - Create a set or array to keep track of visited vertices.
2. **Start with a Vertex:**
 - Choose a starting vertex to begin the DFS traversal.
3. **Explore Adjacent Unvisited Vertices:**
 - Visit the chosen starting vertex.
 - Explore one of its adjacent unvisited vertices.
 - If there are multiple adjacent unvisited vertices, choose one arbitrarily.
4. **Recursively Explore Further:**
 - Once an adjacent unvisited vertex is found, recursively apply the DFS algorithm to that vertex.
 - This involves marking the vertex as visited and exploring its adjacent unvisited vertices.
5. **Backtrack When Necessary:**
 - If there are no more unvisited vertices adjacent to the current vertex, backtrack to

the previous vertex.

- If using recursion, this happens automatically as the function calls return.

6. Repeat Until All Vertices are Visited:

- Continue the process of exploring adjacent unvisited vertices and backtracking until all vertices have been visited.

7. Finish:

- Once all vertices have been visited, DFS traversal is complete.

8. Optional:

- If there are disconnected components in the graph (i.e., vertices that cannot be reached from the starting vertex), repeat steps 2-7 for each unvisited vertex until all vertices are visited.

Example

Define a Tree Node class

class TreeNode:

def __init__(self, value):

self.value = value

self.children = []

Define DFS function

def depthFirstSearch(tree):

if tree is None:

return

print(tree.value, end=" ")

for child in tree.children:

depthFirstSearch(child)

Construct the tree as per diagram

A

/ | \

B C D

```
# /  
# E  
# /\n# G F
```

```
# Create nodes
```

```
A = TreeNode('A')
```

```
B = TreeNode('B')
```

```
C = TreeNode('C')
```

```
D = TreeNode('D')
```

```
E = TreeNode('E')
```

```
F = TreeNode('F')
```

```
G = TreeNode('G')
```

```
# Build the tree structure
```

```
A.children = [B, C, D]
```

```
B.children = [E]
```

```
E.children = [G, F]
```

```
# Run DFS
```

```
print("Depth-First Search Output:")
```

```
depthFirstSearch(A)
```

```
Depth-First Search Output:  
A B E G F C D
```

Explanation

The function then loops over each child of the current node. For each child, it recursively calls the same function, which means it goes deep into the tree before moving to the next and so on. In this traversal would be like below flow:

- ☐ Start at A
- ☐ Go to B
 - Visit B's child E
 - Then visit F
- ☐ Return to A and visit C
- ☐ Then visit D

Task:

1. Write a Python program for the breath first search and depth first search for given graph.

